

Technical Approach

Reference vs. KoylaNiti - A Working Comparison

SIH26024 · What the reference “Slide 3 — Technical Approach” describes, what KoylaNiti actually implements, and what changed as a result of this review.

For internal reference when explaining the technical approach to judges — not a replacement for the reference slide, a working document to reconcile it against the real build.

1. The Reference Material

The two images below are the reference “Slide 3 — Technical Approach” provided for comparison: a presenter script describing a five-layer architecture, and the corresponding slide diagram. Everything in Section 2 onward measures KoylaNiti against these two images, layer by layer.

SLIDE 3 — Technical Approach

Presenter 3 • ~1 minute 30 seconds

ROUGH SCRIPT

Our technical approach follows a **five-layer architecture**.

At the field level, we use a **React, TypeScript and Vite PWA** with Workbox and IndexedDB for offline-first capture. The field worker can capture GPS, timestamp and photo evidence even without connectivity, and the data waits locally until it can sync.

The second layer is the **API and workflow layer**, built with Node.js and TypeScript, which manages obligation and corrective-action flows and escalation rules.

The third layer is **data and evidence**, using PostgreSQL and object storage for photos, documents and audit events.

The fourth is **intelligence**: human-reviewed OCR, deterministic rules, and anomaly detection. Importantly, AI and ML only assist prioritisation; they do not make autonomous legal decisions.

Finally, the **control tower** provides GIS and role-based views from mine level up to subsidiary and oversight levels.

For audit integrity, we also use an append-only, hash-linked audit log, where each event is linked to the previous event using SHA-256, allowing integrity breaks to be detected.

Important if asked: Emphasise “rules first + human verification” and “no autonomous legal decisions.”

Handoff: “With this architecture in place, the next question is whether the solution is practical to deploy and scale.”

Reference image 1 - presenter script.



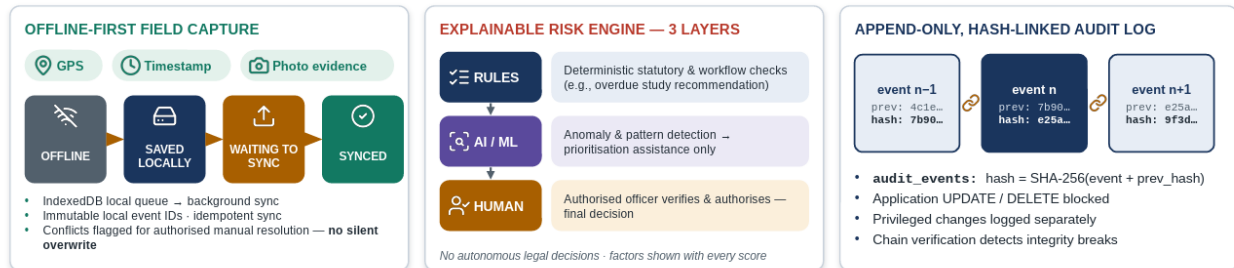
TECHNICAL APPROACH



TECHNOLOGIES TO BE USED · 5-layer architecture



METHODOLOGY & PROCESS OF IMPLEMENTATION



@SIH Idea submission- Template

3

Reference image 2 - slide diagram (five layers, methodology, audit log detail).

2. At a Glance

KoylaNiti deliberately does not adopt the reference's exact stack (Vite/React field app, Node.js API, object storage) - that was an explicit decision, not an oversight, and this review does not change it. What this review does do is check every *capability* the reference slide promises against what KoylaNiti actually does today, and close the gaps that were genuinely missing rather than just differently built.

Layer / Item	Where KoylaNiti stands	Verdict
Field PWA	Stack differs (kept); installability/offline-shell gap - CLOSED this review	MATCHED
API + Workflow	Stack differs (kept); every described behaviour already implemented	MATCHED
Data + Evidence	Postgres matches; object storage vs. DB blob - acknowledged, not changed	PARTIAL
Intelligence - rules	Deterministic KoylaNiti Controls engine - exact match	MATCHED
Intelligence - OCR	Text extraction is human-reviewed; no image OCR step yet	PARTIAL
Intelligence - anomaly ML	Satellite anomaly is seeded demo data, not a live model	GAP
Control Tower	Leaflet (reference lists it as an option) + jurisdiction RBAC	MATCHED
Integration-ready	API is versioned/OpenAPI-documented; no adapter built (none needed yet)	N/A YET
RBAC	Server-enforced role + jurisdiction scoping throughout	MATCHED
Append-only hash-linked audit log	SHA-256 chain + verify endpoint - implemented this review cycle	MATCHED
Encryption at rest	Passwords hashed, transport encrypted; no field-level at-rest encryption	PARTIAL
DB-level UPDATE/DELETE block on audit log	App never issues one; no DB-role REVOKE (needs prod DB access)	PARTIAL
Offline queue: immutable IDs, idempotent sync	Client-generated ids, retry-safe - exact match	MATCHED
No silent overwrite on conflict	Not applicable - inspections are create-only, no concurrent-edit path exists	N/A

3. Layer-by-Layer Detail

3.1 Field Layer - PWA and Offline Capture

Reference: React + TypeScript + Vite PWA, Workbox, IndexedDB. GPS/timestamp/photo capture works offline and syncs when connectivity returns.

KoylaNiti: Next.js 16 (React + TypeScript) instead of Vite, and a **localStorage**-backed offline queue instead of IndexedDB - both deliberate, kept simplifications appropriate at this data volume (a handful of queued inspections with one photo each; IndexedDB earns its complexity at a much larger scale). Every queued inspection carries a client-generated `client_id`, so a retried sync cannot double-submit - matching the reference's "immutable local event IDs, idempotent sync" point exactly, just without IndexedDB underneath it.

What changed this review: until now the app had no manifest and no service worker at all - it captured data offline via the queue, but could not be installed to a home screen and would not load its own shell with zero connectivity, which is squarely what the reference's "Field PWA" layer promises. This review adds `manifest.json`, app icons, and a small hand-written service worker (network-first with cache fallback for same-origin GETs; it never touches non-GET requests or any `/api/` call, so it can't collide with the existing offline-queue logic) that precaches the inspector shell (`/inspector`, `/inspector/report/new`). Verified in-browser: the manifest resolves, the service worker registers and activates, and the shell routes are present in the cache after one visit.

3.2 API + Workflow Layer

Reference: Node.js + TypeScript managing obligation/corrective-action flows and escalation rules.

KoylaNiti: FastAPI + SQLAlchemy 2.0 (Python) instead of Node.js - a runtime choice, not a capability gap. The case lifecycle is an explicit state machine (DETECTED → TRIAGED → ASSIGNED → INSPECTION_REMEDIATION → EVIDENCE_SUBMITTED → VERIFIED → CLOSED) with server-enforced allowed transitions, and SLA due-dates with an explicit escalation target field drive the notification/escalation logic. Every behaviour the reference names is present; only the language runtime differs, and that was explicitly out of scope for this review to change.

3.3 Data + Evidence Layer

Reference: PostgreSQL plus object storage for photos, documents, and audit events.

KoylaNiti: PostgreSQL - exact match. Photos and documents are stored as binary columns (bytea) directly in Postgres rather than a separate object store (S3/GCS/MinIO). This is an honest, acknowledged simplification: fine at hackathon-demo data volumes, but a real deployment would move large binary evidence out of the relational database before it hurts backup size and query performance. Not changed in this review - swapping storage backends is infrastructure work that needs real cloud storage credentials the user would provision themselves, and it changes nothing about what a judge sees in a demo today. This is the clearest, most honest "what we'd do with more time" answer for this layer.

3.4 Intelligence Layer

Reference: human-reviewed OCR, deterministic rules, and AI/ML anomaly detection - explicitly "assist prioritisation, never autonomous legal decisions."

Deterministic rules - exact match: the KoylaNiti Controls engine is a versioned rule table (never a hardcoded if-check) driving every score. **OCR** - partial: documents are entered as text rather than uploaded as scanned images/PDFs, so there's no image-to-text OCR step; what does exist is a local Ollama/Qwen model extracting structured fields (issue date, expiry date, permit number) from that text, with every extracted field shown to and editable by the officer before anything is trusted - the "human-reviewed" principle is honoured even without a literal OCR pass. This is an already-documented, known gap (the manual's Failure Handling section names it),

not something newly discovered here.

Anomaly/pattern detection - this is the one genuine conceptual gap against the reference. Satellite monitoring shows a real before/after image pair with an NDVI-loss percentage and an anomaly polygon per mine, but it is static seeded demo data, not a model that runs against new imagery. The three real AI calls in the app (document field extraction, case-brief generation, complaint summarisation) are all text/NLP tasks, not anomaly detection over imagery or time-series data. This was judged out of scope to fake or rush into this review: standing up a genuine anomaly-detection model needs real satellite time-series data to train or threshold against, which doesn't exist in a hackathon seed dataset - building a hollow version would be worse than being direct about it being future work. **Talking point:** everything the app currently calls "AI" already follows the reference's own rule - it assists (extraction, summarisation, explaining a score) and never decides - it's just applied to text evidence today rather than imagery.

No autonomous legal decisions - true throughout: a case never auto-closes or auto-verifies; every transition beyond the scoring engine's own deterministic DETECTED trigger requires an authenticated officer action, and closure has its own manual verification gate.

3.5 Control Tower

Reference: Leaflet/MapLibre GIS with RBAC views from mine level up to subsidiary and oversight levels.

KoylaNiti: Leaflet - a full match, since the reference itself lists Leaflet as an accepted option alongside MapLibre. RBAC is Field Inspector (assigned cases + own reports) → Mine Manager (their one mine) → DGMS Officer (their state jurisdiction) → Ministry Admin (everything). The terms differ from the reference's generic "subsidiary" language, but the shape is the same - and jurisdiction-by-state is arguably the more accurate fit for how Indian coal-mine oversight (DGMS) actually works, versus a generic corporate-subsidiary model.

3.6 Integration-Ready

Reference: "existing systems → future API adapters."

KoylaNiti: every capability is exposed through one versioned REST API (/api/v1/. . .) with FastAPI's auto-generated OpenAPI docs at /docs - the natural seam for a future adapter into a real DGMS or state pollution-control-board system. No adapter has actually been built, because there is no real external system to adapt into yet inside a hackathon prototype. This is a roadmap/positioning point rather than a component to build today, and is called out as "already positioned for, not yet built" rather than left unmentioned.

3.7 Security & Deployment

Reference: RBAC, encryption, append-only audit events, cloud-agnostic deployment.

- **RBAC** - exact match, enforced server-side via a composable dependency chain, never trusted from the client.
- **Append-only audit events** - matched and, as of this review cycle, strengthened beyond a plain append-only table into a SHA-256 hash-linked chain (see 3.9) - the same feature the reference names in its own dedicated box, including the same core formula.
- **Cloud-agnostic deployment** - largely true: FastAPI + Postgres + Next.js has no vendor lock-in; the deployment docs target Render + Neon + Vercel today but swap cleanly for any other host, managed Postgres, or Node-hosting provider.
- **Encryption** - partial, honestly: passwords are bcrypt-hashed and transport is encrypted once deployed behind Vercel/Render's TLS, but there is no field-level at-rest encryption on sensitive columns. Implementing that meaningfully needs a concrete threat model (which fields, which key-management approach) rather than a superficial checkbox, so it's named here as a next step rather than something coded into this review.
- **DB-level UPDATE/DELETE block on the audit table** - the application code never issues one (true today), but there is no database-level privilege revoke stopping a privileged Postgres role from editing rows directly - already disclosed in the manual and on the audit-integrity page itself. Fixing this for real means creating a

dedicated low-privilege application DB role and revoking UPDATE/DELETE on that table, which needs the database superuser's own hands (this review will not type or enter database credentials under any circumstance, and that boundary was already exercised once this session when the password was offered directly).

3.8 Offline-First Field Capture Methodology

Reference: GPS/timestamp/photo capture; local queue → background sync; immutable local event IDs and idempotent sync; conflicts flagged for manual resolution, never silently overwritten.

KoylaNiti: GPS, timestamp, and photo capture on every field report - exact match. Local queue → sync - matched, via localStorage rather than IndexedDB (see 3.1). Immutable local IDs / idempotent sync - exact match via client-generated IDs. "No silent overwrite" does not apply in its literal sense here: the reference is guarding against a field worker's local edit clobbering someone else's server-side edit to the *same* record. KoylaNiti's inspections are create-only - a field report is submitted once and never edited by a second person - so that specific conflict scenario cannot structurally occur. Stated explicitly so it reads as "not applicable by design," not as an overlooked gap.

3.9 Append-Only, Hash-Linked Audit Log

Reference: every audit event links to the previous one via SHA-256 (`hash = SHA-256(event + prev_hash)`), so integrity breaks are detectable; application UPDATE/DELETE blocked; privileged changes logged separately; chain verification exposed.

KoylaNiti: implemented to match, ahead of this specific request - every audit row stores `prev_hash` and its own hash, computed as `sha256(event_type|mine_id|case_id|user_id|detail|prev_hash)`. A `GET /api/v1/audit/verify` endpoint recomputes the entire chain and reports either a clean bill of health or the exact event id where it breaks; a new `/gov/audit` page surfaces this to DGMS officers and Ministry Admins in plain language, and every audit entry in a case's evidence graph shows its own hash fragment with a link to verify the whole chain. Deliberately tested by tampering with a row directly through the database and confirming the verify endpoint both flags it and names the exact broken event. The one remaining honest limit, stated on the page itself: this detects tampering after the fact - it does not prevent a privileged database user from editing rows directly, since there is no database-level trigger (see 3.7's DB-role note).

4. Changes Made As a Result of This Comparison

Two concrete changes came directly out of reconciling KoylaNiti against this reference slide - both additive to the existing Next.js/FastAPI/Postgres approach, neither a rewrite of it:

- **Append-only, hash-linked audit log** (backend: `app/audit.py`, `app/routers/audit.py`; frontend: `/gov/audit`, the evidence graph's audit timeline). Replaced a plain, honestly-limited audit table with a SHA-256 hash chain and a verify endpoint, matching the reference's dedicated audit-log box directly.
- **Installable, offline-capable Field PWA** (`app/public/manifest.json`, `app/public/sw.js`, registered from the inspector layout). Closed the gap between "data capture works offline" (already true, via the `localStorage` queue) and "the app itself installs to a home screen and loads its shell with zero connectivity" (previously not true, now verified working).

Everything else in this document is a *documented* difference, not a coded change - either a deliberate, kept stack decision (Next.js vs. Vite, FastAPI vs. Node.js, `localStorage` vs. IndexedDB, Postgres blobs vs. object storage), a genuine future-work item that needs data or infrastructure this review does not have (a real anomaly-detection model, true OCR over scanned documents, field-level encryption, a DB-level privilege revoke), or a point that doesn't apply to KoylaNiti's actual data model (the silent-overwrite conflict scenario).

5. Talking Points for Judges

- "We use the same five capabilities the reference architecture describes - offline field capture, a workflow API, an evidence store, a rules-plus-AI intelligence layer, and a role-based control tower - implemented on a stack we chose for reasons specific to this problem (Next.js Server Actions removed an entire client-fetch layer; FastAPI's typed dependency injection is what makes our RBAC actually enforced server-side, not just client-hidden)."
- "Our AI is deliberately narrow and human-reviewed: it extracts fields and drafts summaries, never auto-closes a case or auto-trusts an extraction. That's the same 'rules first, AI assists, human decides' principle the reference architecture calls for - we just don't have a live anomaly-detection model behind our satellite view yet, and we'd rather say that plainly than claim it."
- "Our audit log is append-only and hash-linked exactly the way the reference describes - SHA-256, chained, independently verifiable, and we've tested it against a real tampering attempt."
- "The field app is now a real installable PWA with an offline shell, on top of an offline data queue that already worked - so field inspectors can open and use it with zero signal, not just queue data while already inside it."
- "Where we have honest gaps - object storage, field-level encryption, real OCR, a live anomaly model - we know exactly what each one needs and why we didn't fake it for a demo."